

PROPUESTA DE VIDEOJUEGO — CRÓNICAS DEL TIEMPO (DISEÑO PARA C++/QT)

1. Sinopsis general

Crónicas del Tiempo es un videojuego sencillo diseñado para ser implementado en C++ con Qt (QGraphicsView/QGraphicsScene), compuesto por tres niveles, cada uno inspirado en un acontecimiento histórico distinto. El objetivo del jugador es superar los retos particulares de cada escena ajustándose a diferentes dinámicas, físicas y tipos de cámara. El juego busca ser corto, agradable a la vista y con dificultad progresiva pero justa. La entrega actual corresponde únicamente a la propuesta de diseño (sin programación).

2. Objetivos de aprendizaje del curso

- Aplicar principios de Programación Orientada a Objetos (POO) en el modelado de niveles, entidades y físicas.
- Planear el uso de memoria dinámica segura (punteros inteligentes) para la futura implementación.
- Diseñar la interfaz gráfica (GUI) con Qt y un bucle de juego basado en QTimer (planificado).
- Definir y parametrizar al menos tres modelos físicos (ecuaciones) distribuidos entre los niveles.
- Incorporar un agente autónomo con percepción, razonamiento, acción y aprendizaje simples.

3. Visión creativa y alcance

La propuesta abarca tres niveles breves e independientes, cada uno con identidad visual y mecánicas diferenciadas. Se evitarán mecánicas de plataformas; en cambio, se usarán esquemas

de desplazamiento lateral sin saltos entre plataformas, vista cenital con scroll y control de caída/aterrizaje vertical. Al menos un nivel estará controlado por tiempo.

4. Niveles propuestos (temática histórica 2025-2)

Nivel 1 — El Mensajero de Maratón (490 a. C.)

- Vista: lateral fija (no plataformero).
- Contexto histórico (sinopsis): inspirado en la legendaria carrera del mensajero griego tras la batalla de Maratón.
- Dinámicas principales:
 - Gestión de resistencia; el jugador regula el ritmo para evitar agotarse.
 - Esquiva de proyectiles (flechas) con perturbaciones por viento.
 - HUD con temporizador: el reto es llegar antes de que el tiempo expire.
- Retos y objetivos: llegar a la meta con vida y dentro del límite temporal.
- Físicas (parametrizables):
 - 1) Movimiento amortiguado del corredor: $x(t+\Delta)=x(t)+v_x\Delta$; $v_x(t+\Delta)=v_x(t)+a_x\Delta-c\cdot v_x\Delta$
 - 2) Trayectoria de flecha oscilatoria: $x(t)=x_0+A\cdot\sin(\omega t+\phi)$; $y(t)=y_0+v_y\cdot t$
 - 3) Perturbación de viento: $\Delta x_v = k\cdot\sin(2\pi f\cdot t)$
- Funcionamiento general: cámara fija lateral, QTimer (futuro) para actualización, indicadores de vida/tiempo/estamina.

- Este nivel cumple el requisito de control por tiempo.

Nivel 2 — Caravana en la Ruta de la Seda (siglo XIV)

- Vista: cenital con scroll limitado (la cámara sigue al jugador; mapa con rutas).
- Contexto histórico (sinopsis): travesía mercantil entre puestos de comercio en Asia central.
- Dinámicas principales:
 - Terrenos con diferentes coeficientes de arrastre (arena, camino, roca).
 - Tormentas que generan campos de viento no uniformes.
 - Agente autónomo (bandido) que patrulla y aprende rutas frecuentadas por el jugador.
- Retos y objetivos: alcanzar sucesivos puestos sin ser interceptado y optimizando trayectorias.
- Físicas (parametrizables):
 - 1) Movimiento con arrastre: $v(t+\Delta)=v(t)+a\Delta-k\cdot v(t)\Delta$
 - 2) Campo de viento: $w(x,y,t)=\langle a\cdot\sin(\omega t+\alpha x), b\cdot\sin(\omega t+\beta y)\rangle$
 - 3) Partículas de arena con pequeñas parábolas para efecto visual y retroalimentación.
- Funcionamiento general: navegación libre, decisiones tácticas frente a clima y rutas; HUD de distancia/alerta de avistamiento.

Nivel 3 — Alunizaje 1969

- Vista: vertical tipo ‘lander’ (no plataformas).

- Contexto histórico (sinopsis): descenso controlado de un módulo lunar inspirado en la misión Apolo 11.

- Dinámicas principales:

- Gestión de combustible vs. empuje.
- Control de orientación y velocidad de descenso.
- Zona de aterrizaje con tolerancias estrictas.

- Retos y objetivos: posar el módulo dentro de umbrales de $|v_y|$, $|v_x|$ y $|\theta|$ con combustible limitado.

- Físicas (parametrizables):

- 1) Gravedad + empuje: $a = \langle (T/m) \cdot \cos\theta, (T/m) \cdot \sin\theta - g \rangle$

- 2) Rotación con inercia: $\theta(t+\Delta) = \theta(t) + \omega\Delta$; $\omega(t+\Delta) = \omega(t) + \tau/I \cdot \Delta$

- 3) Colisión parcialmente elástica (rebotar suave si se excede levemente la velocidad).

- Funcionamiento general: control fino de motores, feedback visual/auditivo, medidor de combustible y vector velocidad.

5. Agente autónomo (componente investigativo)

Se incorpora en el Nivel 2 como un ‘bandido’ que patrulla y reacciona al jugador.

- a) Percepción: cono de visión y detección por proximidad/ruido (umbral de distancia).

- b) Razonamiento: máquina de estados finitos (Patrullar $\rightarrow$ Perseguir $\rightarrow$ Anticipar $\rightarrow$ Buscar).

c) Acción: moverse, acelerar, interceptar, colocar una trampa temporal en rutas probables.

d) Aprendizaje: mapa de calor 2D (grid); se incrementa en celdas visitadas por el jugador.

Política ϵ -greedy prioriza celdas ‘calientes’. Persistencia prevista con QSettings.

6. Arquitectura propuesta (POO, memoria y GUI)

- Clases principales: Game, LevelBase, Level1/Level2/Level3, Entity, Player, Agent, HUD, PhysicsModel (abstracta) y derivados.
- Polimorfismo: LevelBase define la interfaz común (init, update, reset). Cada nivel implementa su lógica y físicas.
- Composición: Game contiene la vista (QGraphicsView) y la escena actual (QGraphicsScene).
- Memoria dinámica: se planifica el uso de `std::unique_ptr`/`std::shared_ptr` y RAII para entidades y recursos.
- GUI Qt: menús simples, HUD con texto y barras; señales/slots para entradas y timers.

7. Parámetros y balance de dificultad

- Nivel 1: tiempo (60–120 s), tasa de resistencia (c), amplitud/frecuencia del viento (k, f), densidad de flechas.
- Nivel 2: coeficientes de arrastre por terreno (k), intensidad/frecuencias de viento (a, b, ω , α , β), radio de visión del agente.
- Nivel 3: gravedad efectiva (g), empuje máximo (T), masa (m), tolerancias de aterrizaje, combustible inicial.
- Telemetría simple: registrar fallos/éxitos y ajustar parámetros tras tres intentos fallidos (futuro).

8. Plan de pruebas (conceptual)

- Pruebas funcionales: verificación de condiciones de victoria/derrota y HUD coherente.
- Pruebas de jugabilidad: sesiones de 2–3 minutos por nivel, medición de frustración/aburrimiento.
- Pruebas del agente: validar transición de estados y mejora de interceptación a lo largo de partidas simuladas.
- Criterios de aceptación: dificultad progresiva, reglas claras, feedback inmediato.